

Node.js Core da Optimizasyon yapmak

Mert Can Altın

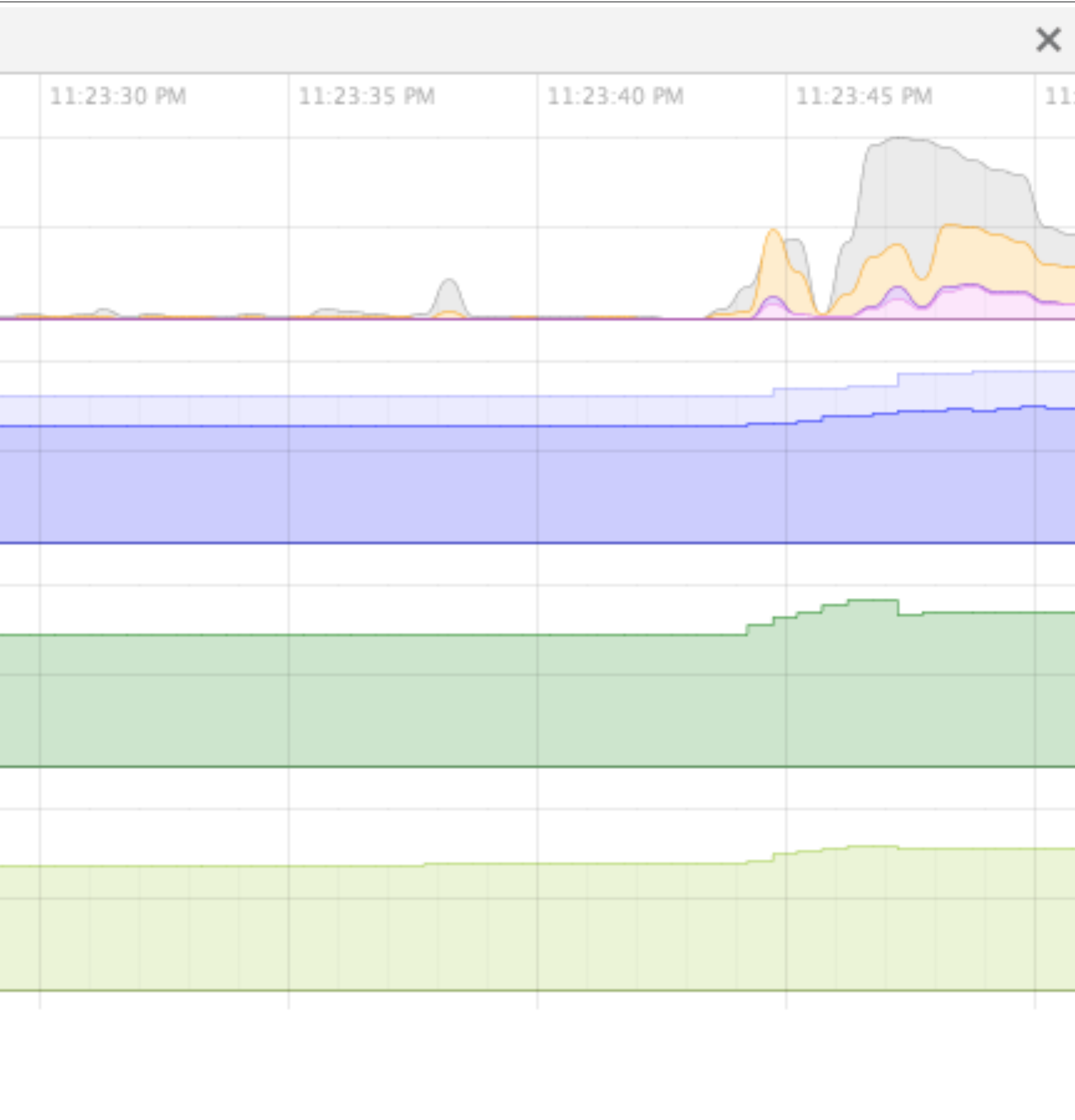
@Trendyol Software Engineer

@Node.js Core Contributor

@OpenJS CPC Member



Bu Sunumda Neler Öğreneceksiniz?



- 1. Optimizasyon Nedir?
- 2. Optimizasyon Alanları nelerdir?
- Optimizasyon sorunları - öneriler
 - Performance issue nasıl tespit edilir?
 - Benchmarklar
 - perf_hook
 - Profile
- TextDecoder Fast Path: Başarı, Hata ve Düzeltme
- TextDecoder Nedir?
 - 5. Hata yaptığınızda ne olur?
 - 6. Community ile nasıl çalışılır?
 - 7. Regression'ları nasıl düzeltirsiniz?

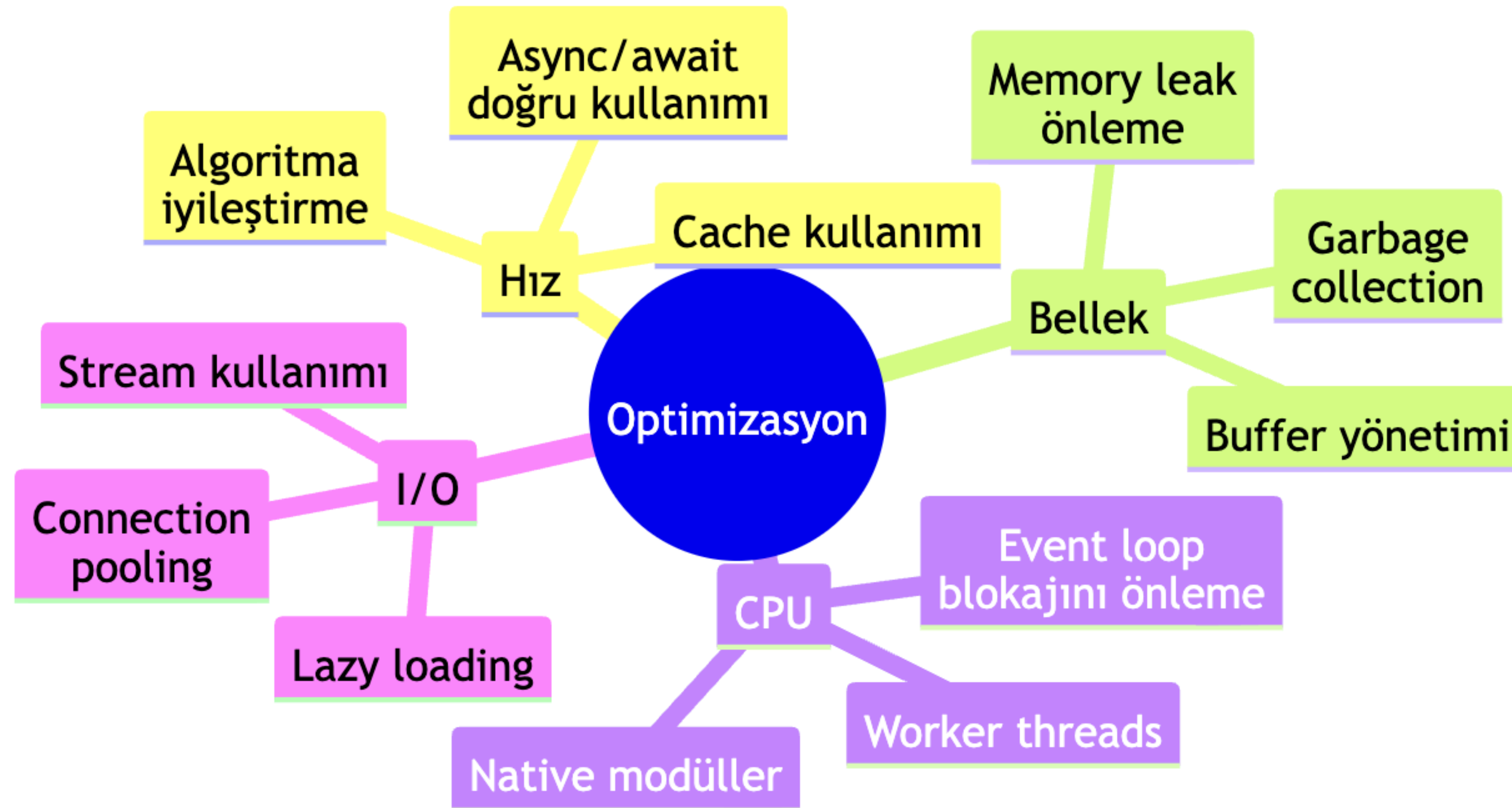
Optimizasyon nedir?

Optimizasyon, bir sistemin, kodun veya sürecin en verimli, en hızlı veya en az kaynak tüketen şekilde çalışmasını sağlamak için yapılan iyileştirmelerdir.

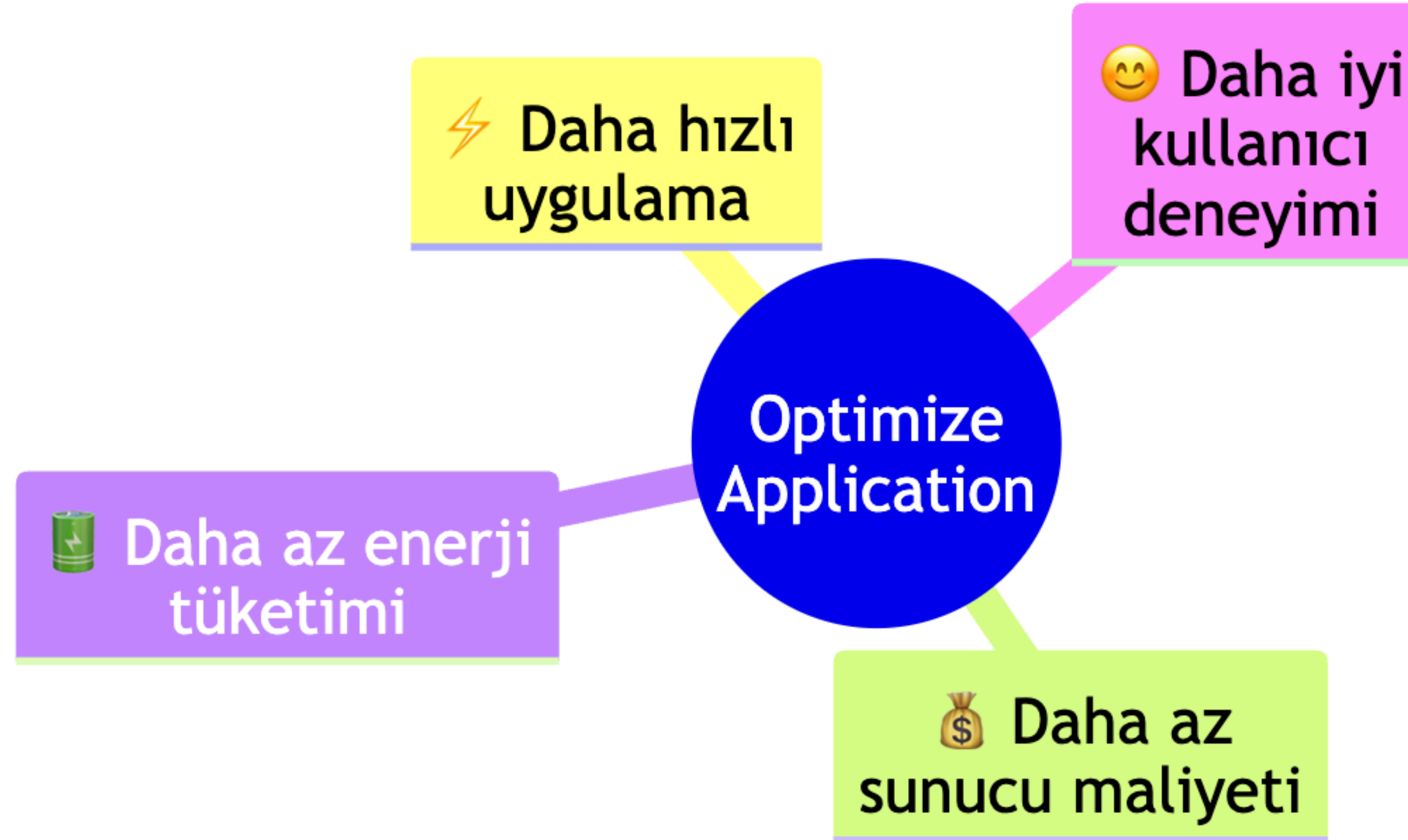
Yazılımda optimizasyon genellikle bu alanlarda yapılır

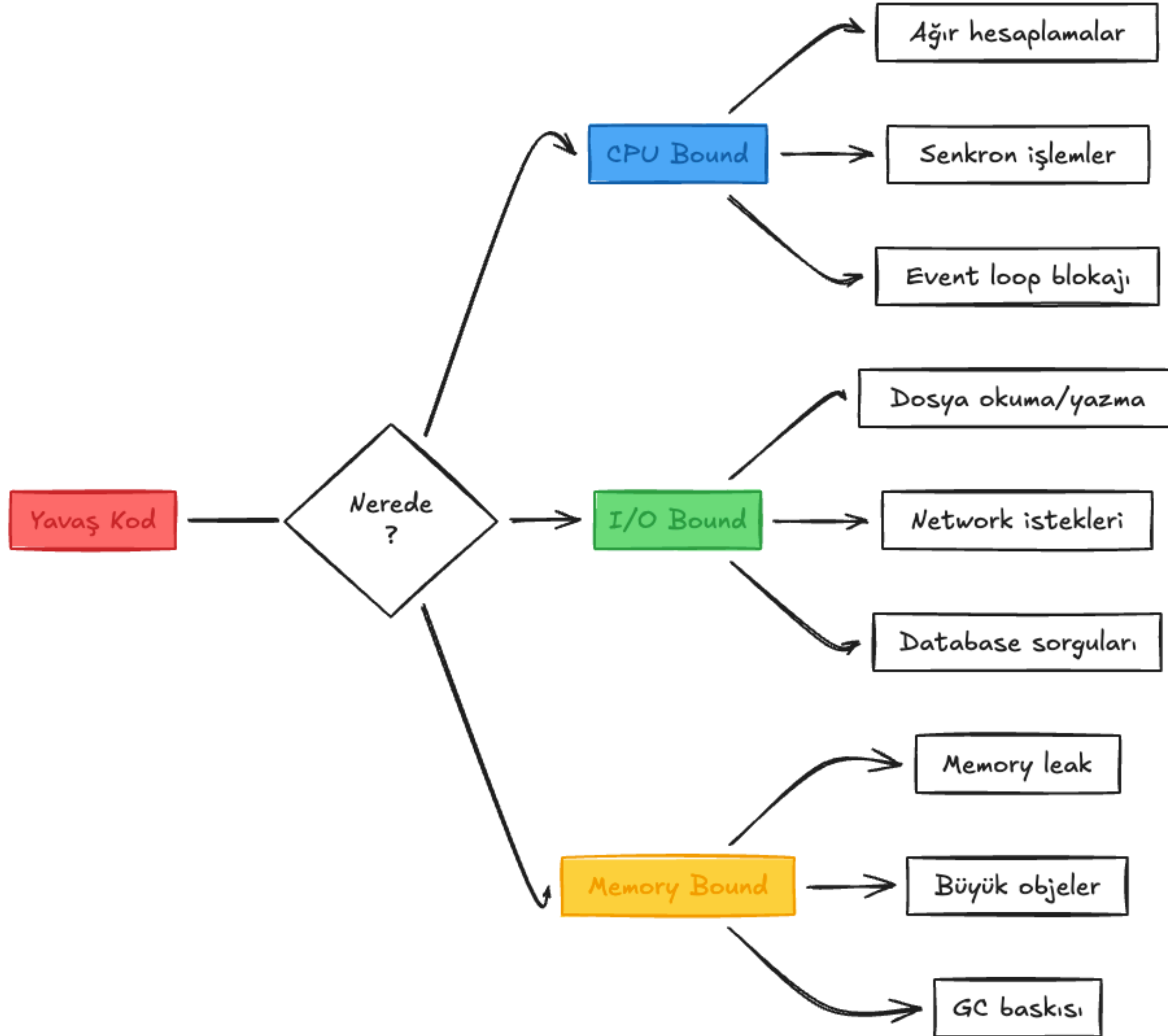
Hız (Performance)	Kodun daha hızlı çalışması
Bellek (Memory)	Daha az RAM kullanımı
CPU	İşlemci kaynaklarının verimli kullanımı
I/O	Dosya/ağ işlemlerinin hızlandırılması
Boyut	Daha küçük bundle/binary dosyaları

Node.js'de Optimizasyon Örnekleri

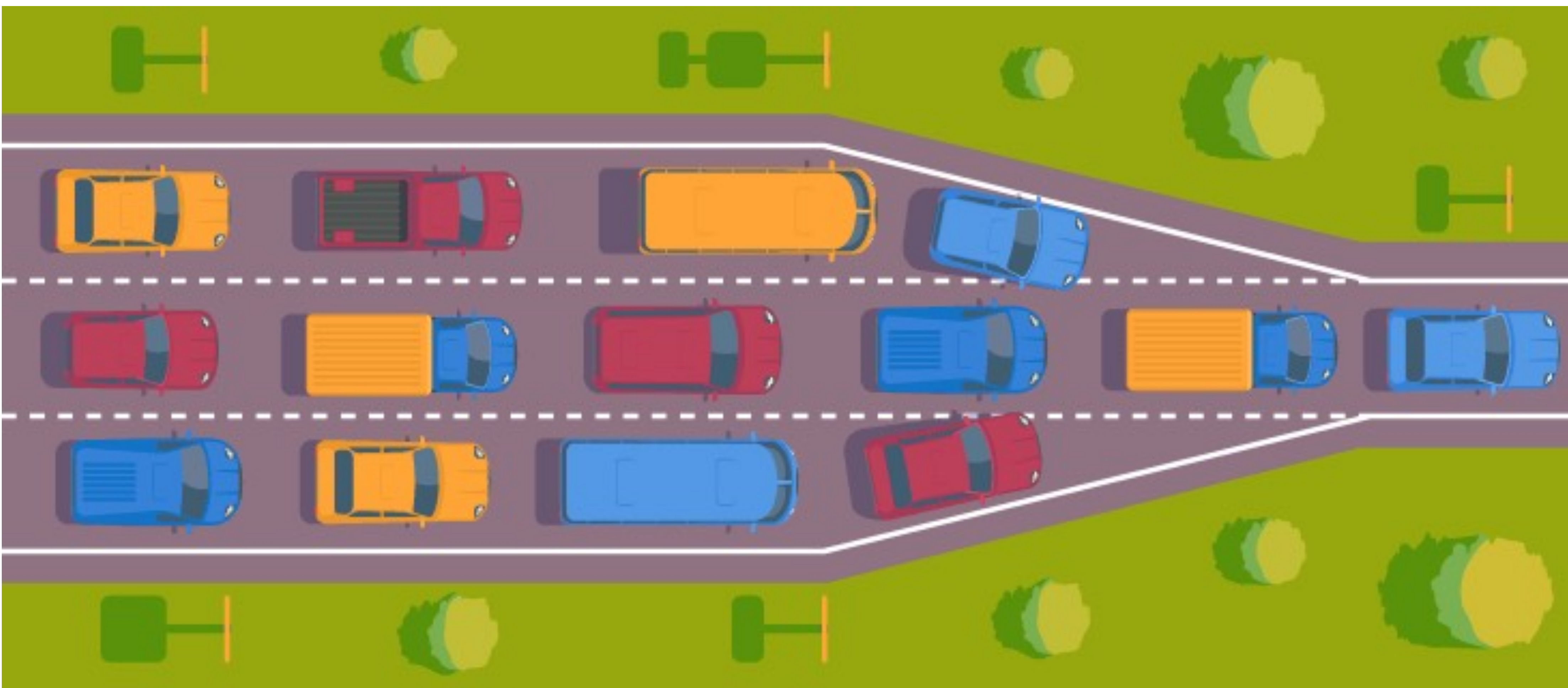


Neden Önemli?



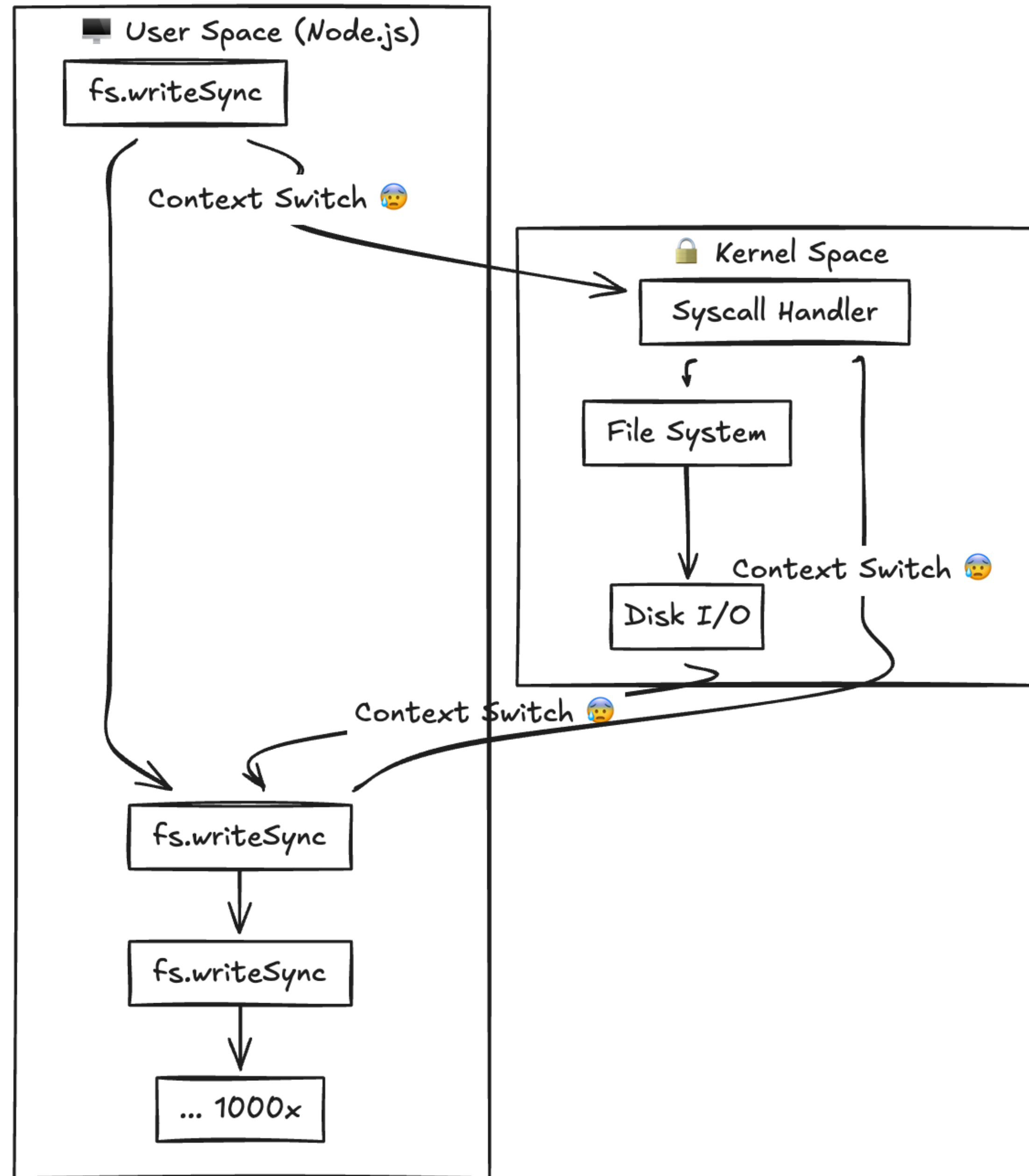


Darboğaz /Bottleneck



Bazı optimizasyon sorunları

Syscall Yükü



System Call Batch'leme

```
// Her satır için ayrı syscall
function writeLineByLine(filename, lines) {
  const fd = fs.openSync(filename, 'w');

  for (const line of lines) {
    fs.writeSync(fd, line + '\n'); // 1000 satır = 1000 syscall
  }

  fs.closeSync(fd);
}
```

System Call Batch'leme

```
const lines = Array(1000).fill('Hello World');  
console.time('Line by line');  
writeLineByLine('test1.txt', lines);  
console.timeEnd('Line by line');
```


❌ 1000 Syscall Sorunu

1000 write()



1000 context switch



~5M CPU cycle



Yavaş I/O

— 1000x fark! ➡

✅ 1 Syscall Çözümü

Buffer biriktir



1 write()



1 context switch



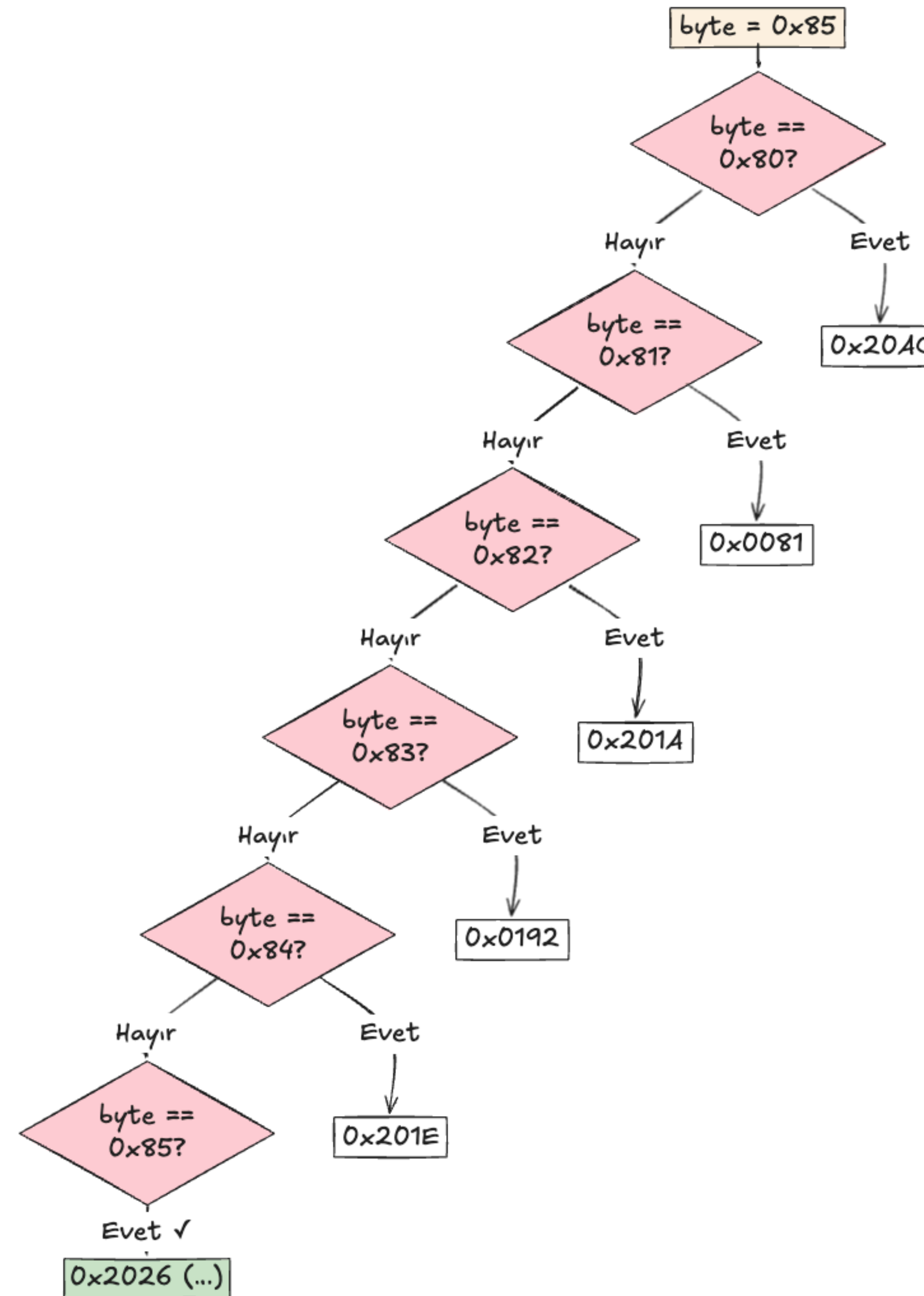
~5K CPU cycle

Lookub Table ve iF Else

Lookup Table

```
function decodeByteIfElse(byte) {  
    if (byte === 0x80) return 0x20AC;      // €  
    else if (byte === 0x81) return 0x0081;  
    else if (byte === 0x82) return 0x201A; // ,  
    else if (byte === 0x83) return 0x0192; // f  
    else if (byte === 0x84) return 0x201E; // „  
    else if (byte === 0x85) return 0x2026; // ...  
    // ... 32 tane if-else 😓  
    else return byte;  
}
```

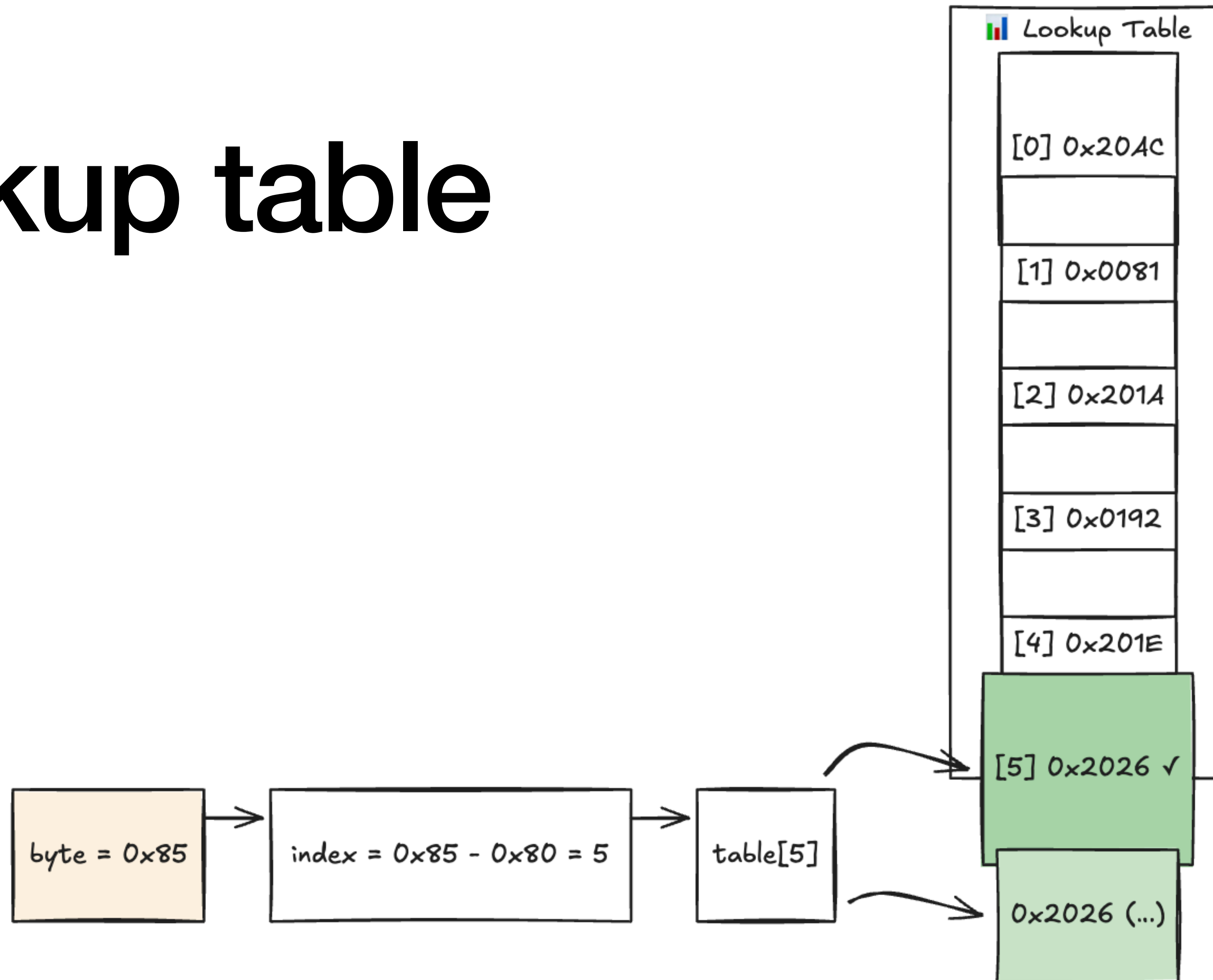

if else



Lookup Table

```
const WINDOWS_1252_TABLE = [  
    0x20AC, 0x0081, 0x201A, 0x0192, 0x201E, 0x2026, 0x2020, 0x2021,  
    0x02C6, 0x2030, 0x0160, 0x2039, 0x0152, 0x008D, 0x017D, 0x008F,  
    0x0090, 0x2018, 0x2019, 0x201C, 0x201D, 0x2022, 0x2013, 0x2014,  
    0x02DC, 0x2122, 0x0161, 0x203A, 0x0153, 0x009D, 0x017E, 0x0178  
];  
  
function decodeByteTable(byte) {  
    if (byte >= 0x80 && byte <= 0x9F) {  
        return WINDOWS_1252_TABLE[byte - 0x80]; // Tek erişim! ✂  
    }  
    return byte;  
}
```

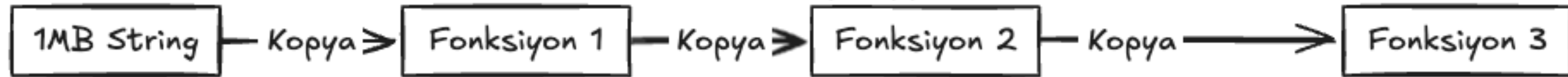

Lookup table



string_view (Zero-copy)

```
void process(std::string data) {  
    // KOPYA oluşur  
}
```


❌ String Kopyalama

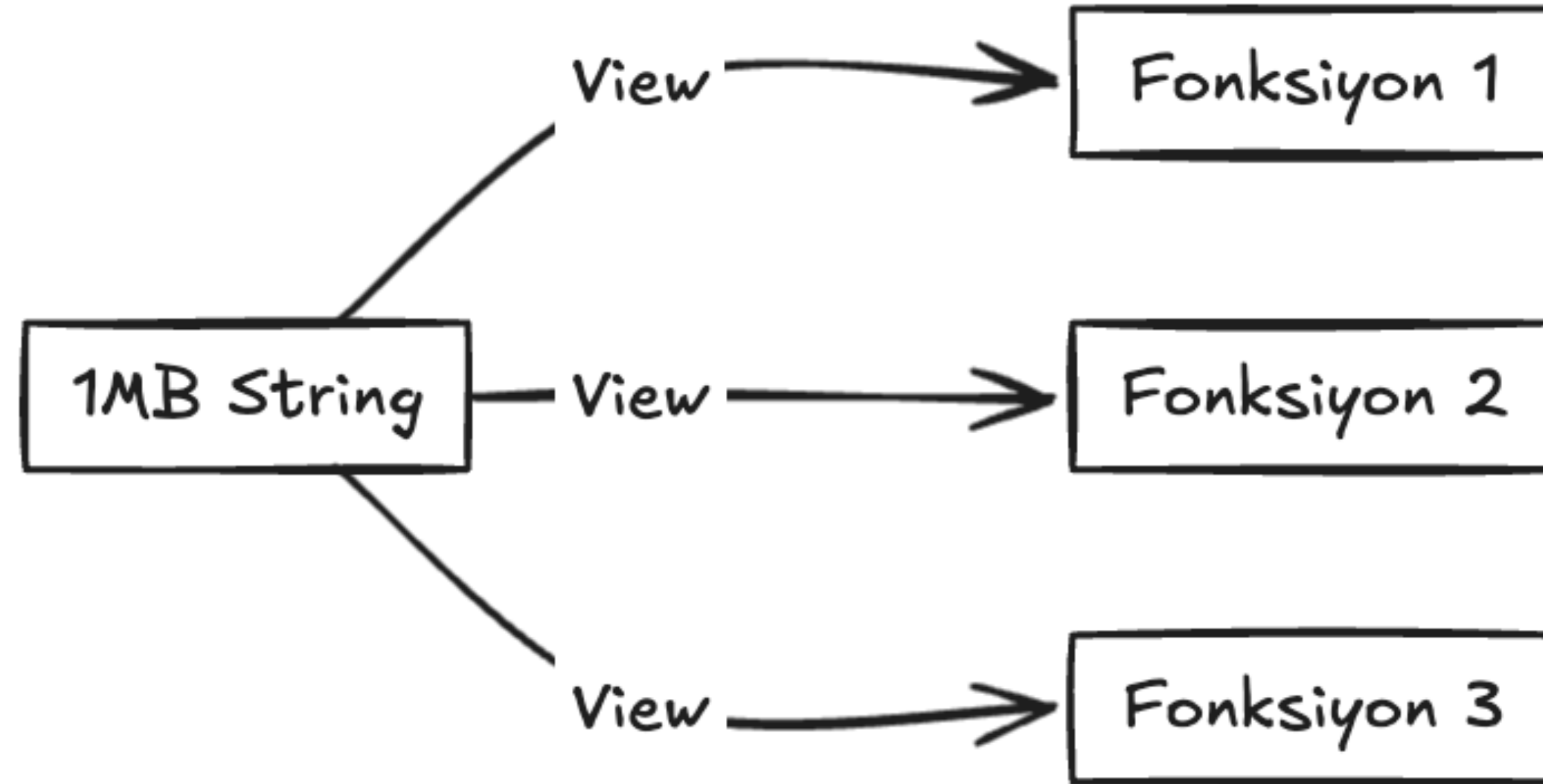


Toplam: 4MB kullanıldı 😓

string_view (Zero-copy)

```
void process(std::string_view data) {  
    // Sadece pointer + size  
}
```

✓ string_view / Buffer.subarray

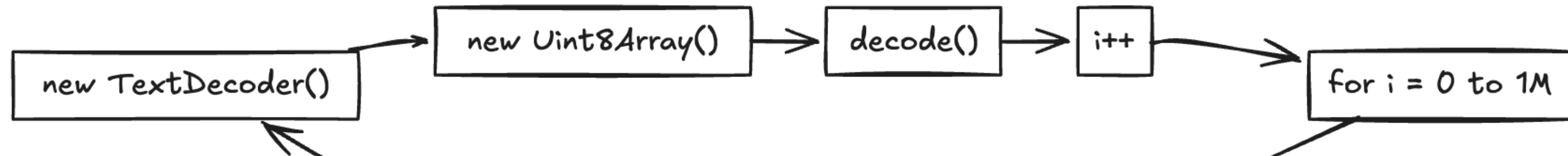


Toplamı: 1MB kullanıldı ⚡

Değişken Reuse

```
function decodeBad(data) {  
    let result = '';  
  
    for (let i = 0; i < 100000; i++) {  
        const decoder = new TextDecoder();           // Her seferinde YENİ  
        const buffer = new Uint8Array([72, 101]);     // Her seferinde YENİ  
        result = decoder.decode(buffer);  
    }  
  
    return result;  
}
```

❌ Loop İçinde Allocation



Sonuç: 2M allocation 🤯

Değişken Reuse

```
function decodeGood(data) {  
  const decoder = new TextDecoder();           // Bir kez ✂  
  const buffer = new Uint8Array([72, 101]);     // Bir kez ✂  
  let result = '';  
  
  for (let i = 0; i < 100000; i++) {  
    result = decoder.decode(buffer);  
  }  
  
  return result;  
}
```


✓ Loop Dışında Allocation

```
const decoder = new TextDecoder()
```

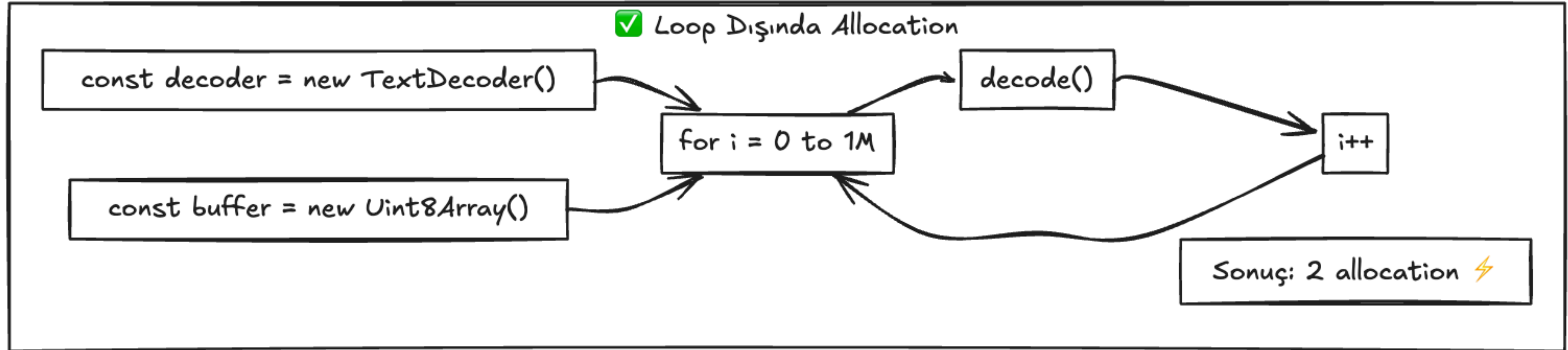
```
const buffer = new Uint8Array()
```

```
for i = 0 to 1M
```

```
decode()
```

```
i++
```

Sonuç: 2 allocation ⚡



Node.js'de Performans Sorunu Tespit Yöntemleri

Yöntem	Kolaylık	Detay	Ne Zaman
<code>console.time</code>	★ ★ ★	★	Hızlı kontrol
<code>perf_hooks</code>	★ ★ ★	★ ★	Hassas ölçüm
<code>--prof</code>	★ ★	★ ★ ★	CPU hotspot
<code>0x</code> / Flame	★ ★	★ ★ ★	Görsel analiz
<code>clinic.js</code>	★ ★ ★	★ ★ ★	Kapsamlı analiz
<code>--inspect</code>	★ ★	★ ★ ★	Interaktif debug
<code>hrtime</code>	★ ★ ★	★ ★	Micro benchmark

Node.js de Benchmark tanımı



Node.js de Benchmark tanımı

Other Top-level files

The top-level files include common dependencies of the benchmarks and the tools for launching benchmarks and visualizing their output. The actual benchmark scripts should be placed in their corresponding directories.

- `_benchmark_progress.js` : implements the progress bar displayed when running `compare.js`
- `_cli.js` : parses the command line arguments passed to `compare.js`, `run.js` and `scatter.js`
- `_cli.R` : parses the command line arguments passed to `compare.R`
- `_http-benchmarkers.js` : selects and runs external tools for benchmarking the `http` subsystem.
- `bar.R` : R script for visualizing the output of benchmarks with bar plots.
- `common.js` : see [Common API](#).
- `compare.js` : command line tool for comparing performance between different Node.js binaries.
- `compare.R` : R script for statistically analyzing the output of `compare.js`
- `run.js` : command line tool for running individual benchmark suite(s).
- `scatter.js` : command line tool for comparing the performance between different parameters in benchmark configurations, for example to analyze the time complexity.
- `scatter.R` : R script for visualizing the output of `scatter.js` with scatter plots.

console.time / console.timeEnd

```
console.time('operation');  
  
// Test edilecek kod  
for (let i = 0; i < 100000; i++) {  
    decoder.decode(data);  
}  
  
console.timeEnd('operation');  
// operation: 234.567ms
```

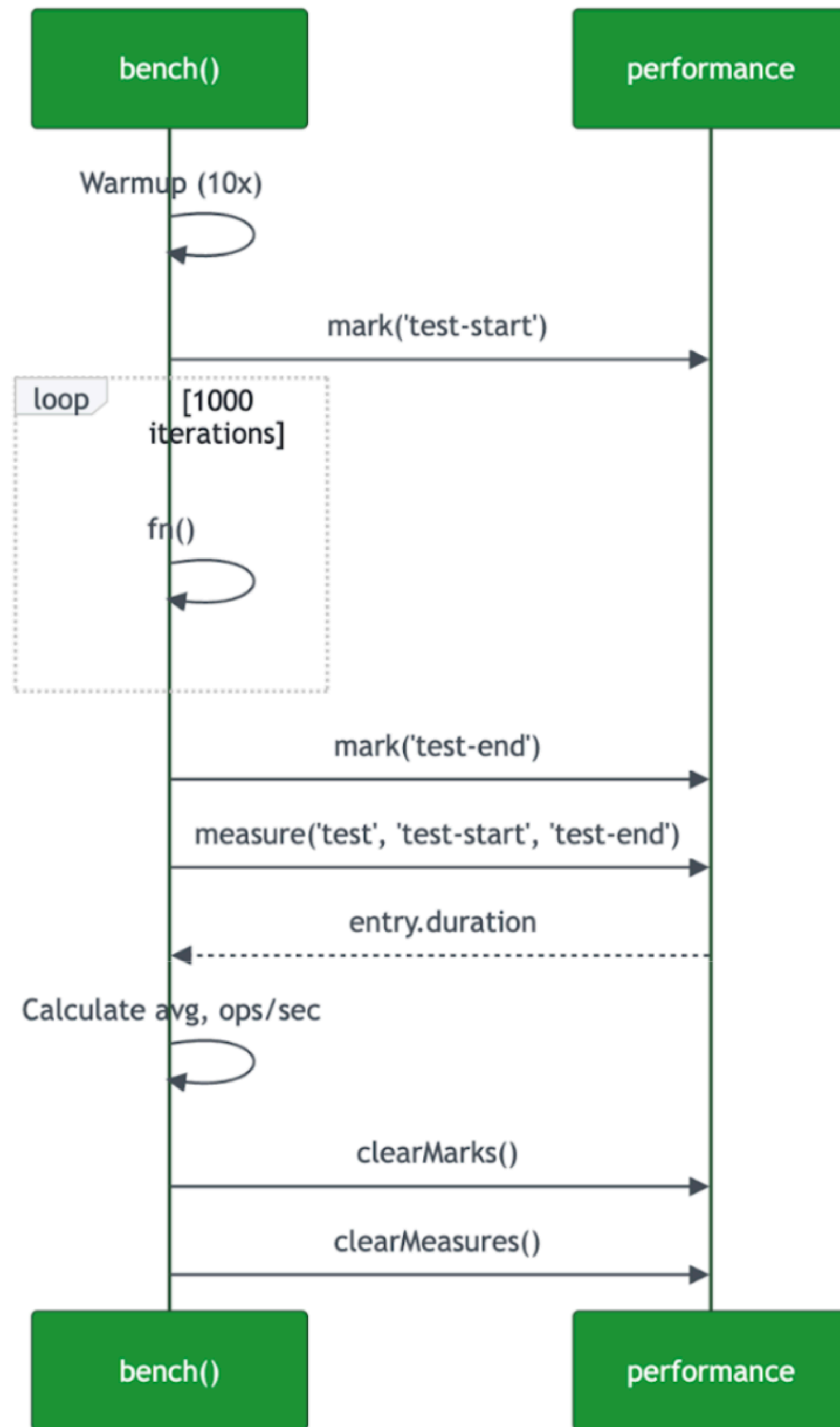

Node.js perf_hooks

```
const { performance, PerformanceObserver } = require('perf_hooks');

// Ölçüm
const start = performance.now();
doSomething();
const end = performance.now();

console.log(`Süre: ${end - start}.toFixed(2)}ms`);

// Veya mark/measure ile
performance.mark('A');
doSomething();
performance.mark('B');
performance.measure('A to B', 'A', 'B');
```



Flame Graph / 0x / Flame

Kurulum

```
npm install -g 0x
```

Çalıştır

```
0x app.js
```

Veya mevcut process'e bağlan

```
0x --open app.js
```

Flame Graph / 0x / Flame Örneği

Abort.ExtraWide [CODE:BytecodeHandler] 18.4% on stack, 18.42% stack top
decode node:internal/encoding:439:11 23.1% on stack, 4.71% stack top
(anonymous) file:///private/tmp/profile-decoder.mjs:1:1 31.4% on stack, 0.27% stack top
Abort.ExtraWide [CODE:BytecodeHandler] 97.1% on stack
Abort.ExtraWide [CODE:BytecodeHandler] 97.1% on stack
~run node:internal/modules/esm/module_job:419:12 97.1% on stack
Abort.ExtraWide [CODE:BytecodeHandler] 98.7% on stack, 1.58% stack top

all stacks

Tiers

Merge

Optimized

Unoptimized

app

deps

core

wasm

inlinable

native

rx

v8

cpp

init

**Performans Sorunu çözerken
yarattığım sorun**

TextDecoder Fast Path: Başarı, Hata ve Düzeltme

Adding a fast path for TextDecoder latin1 #178

[✓ Closed](#)

anonrig opened on Sep 17, 2024

Member



Latin1 still uses ICU, which is extremely slow. We can add a fast path, just like utf8 and make it extremely fast.

Ref: <https://github.com/nodejs/node/blob/main/lib/internal/encoding.js#L424>

cc [@RafaelGSS](#)



UTF-8 fast path'i Latin1 için de uygulayalım!

Öncesi:

- |— ICU kütüphanesi çağrısı
- |— Çok syscall
- |— Yavaş ❌

Sonrası:

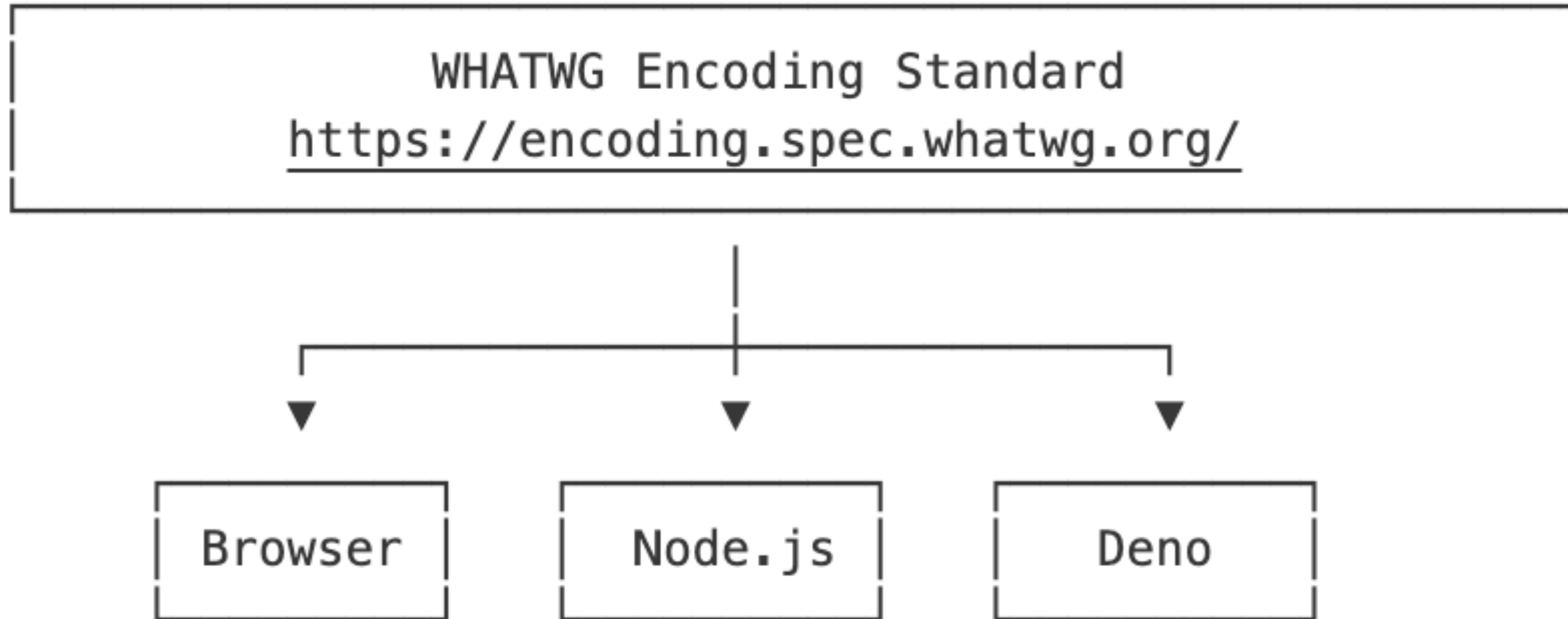
- |— Lookup table (32 eleman)
- |— Direkt memory erişimi
- |— Hızlı ⚡️ ✅

TextDecoder Nedir ve Node.js'deki Yeri

TextDecoder, binary veriyi (Uint8Array, ArrayBuffer) string'e dönüştüren bir Web API'dir.

```
// Basit kullanım
const decoder = new TextDecoder('utf-8');
const bytes = new Uint8Array([72, 101, 108, 108, 111]); // "Hello"
const text = decoder.decode(bytes);
console.log(text); // "Hello"
```

Nereden Geliyor?



Node.js'deki Yeri

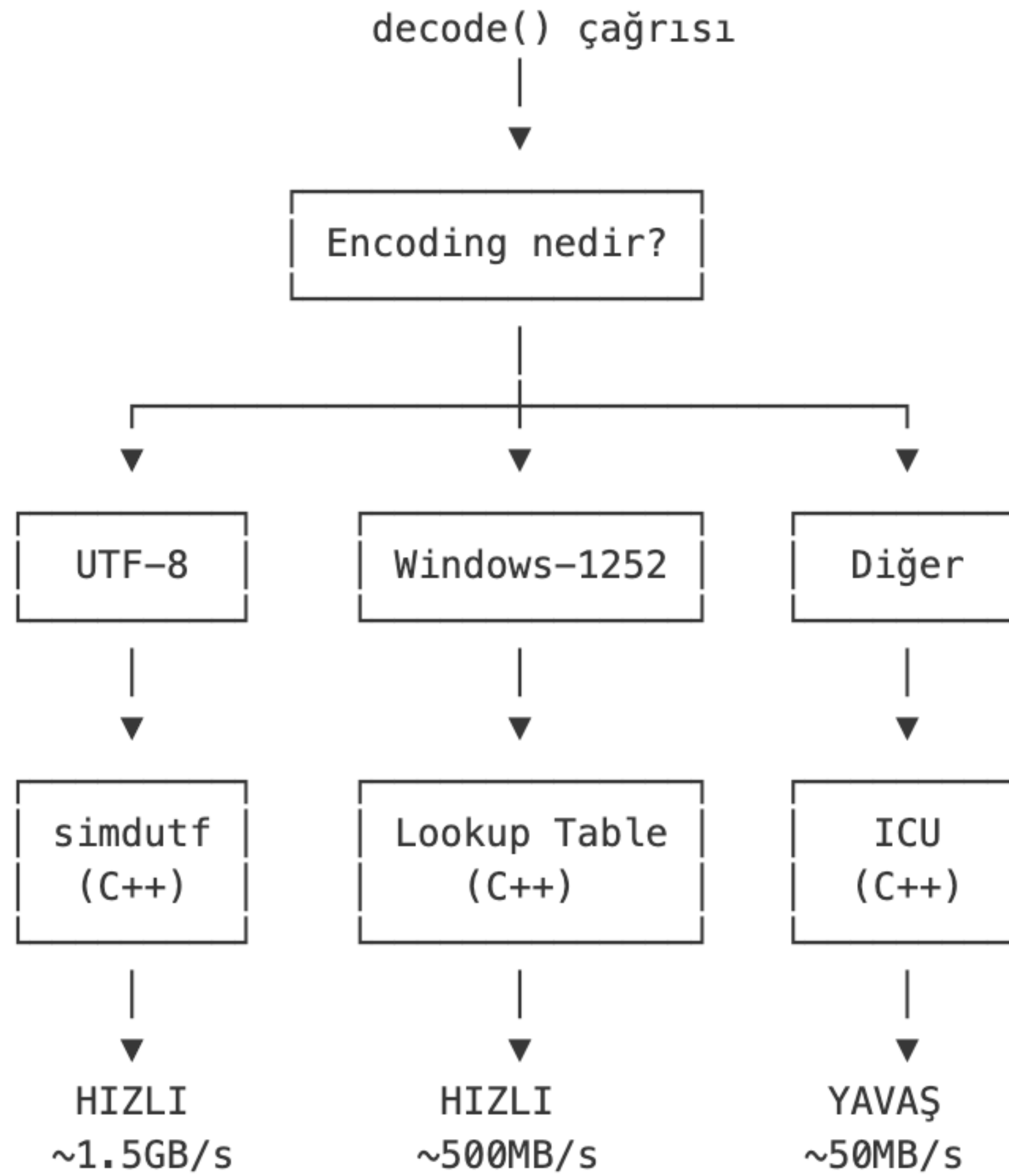
```
Node.js
├── lib/
│   ├── internal/
│   │   └── encoding.js      ← JS implementasyonu
│   └── src/
│       ├── encoding_binding.cc ← C++ native binding
│       └── node_i18n.cc      ← ICU entegrasyonu
└── deps/
    ├── icu-small/           ← Unicode/Encoding desteği
    └── simdutf/              ← Hızlı UTF-8 işlemleri
```


Node.js Implementasyonu

```
// lib/internal/encoding.js
```

```
class TextDecoder {  
  constructor(encoding = 'utf-8', options = {}) {  
    // Encoding'i normalize et  
    // 'latin1', 'ascii' → 'windows-1252' (WHATWG spec)  
    // 'utf8' → 'utf-8'  
  
    this.encoding = encoding;  
    this.fatal = options.fatal || false;  
    this.ignoreBOM = options.ignoreBOM || false;  
  }  
  
  decode(input, options = {}) {  
    // Binary → String dönüşümü  
    // Fast path: UTF-8, Windows-1252  
    // Slow path: ICU (diğer tüm encoding'ler)  
  }  
}
```

Fast Path vs Slow Path



Desteklenen Encoding'ler


```
// Yaygın kullanılanlar
new TextDecoder('utf-8')           // En yaygın
new TextDecoder('utf-16le')        // Windows
new TextDecoder('windows-1252')    // Batı Avrupa
new TextDecoder('iso-8859-1')      // → windows-1252
new TextDecoder('ascii')           // → windows-1252
new TextDecoder('latin1')          // → windows-1252

// Diğer encoding'ler (ICU ile)
new TextDecoder('shift_jis')       // Japonca
new TextDecoder('euc-kr')          // Korece
new TextDecoder('gbk')              // Çince
new TextDecoder('koi8-r')          // Rusça
new TextDecoder('iso-8859-9')      // Türkçe
```

Kullanım Senaryoları

HTTP Response okuma

```
const response = await fetch('https://api.example.com/data');  
const buffer = await response.arrayBuffer();  
const decoder = new TextDecoder('utf-8');  
const text = decoder.decode(buffer);
```


Dosya Okuma

```
const fs = require('fs');  
const buffer = fs.readFileSync('file.txt');  
const decoder = new TextDecoder('windows-1252');  
const text = decoder.decode(buffer);
```

📦 Orijinal Durum

`TextDecoder('windows-1252')`



ICU Library çağrısı



Yavaş decode 🥵

TextDecoder Latin1 için yavaş

WHATWG Encoding Standard'a göre:

latin1 = windows-1252

iso-8859-1 = windows-1252

ascii = windows-1252

Hepsi aynı şey!" dedim... 🤔

Ve bir pr açtım

Yaptıklarımı inceleyelim

<https://github.com/nodejs/node/pull/55275/files>

Encoding API Encodings in Mozilla docs

```
"ansi_x3.4-1968", "ascii", "cp1252", "cp819", "csiso"latin1",  
"ibm819", "iso-8859-1", "iso-ir-100", "iso8859-1", "iso88591",  
"iso_8859-1", "iso_8859-1:1987", "l1", "latin1", "us-ascii",  
"windows-1252", "x-cp1252"
```

[windows-1252](#) ↗

Gözümde kaçan nokta

Note

The [windows-1252 encoding](#) has various [labels](#), such as "latin1", "iso-8859-1", and "ascii", which have historically been confusing for developers. On the web, and in any software that seeks to be web-compatible by implementing this standard, these are synonyms: "latin1" and "ascii" are just labels for [windows-1252](#), and any software following this standard will, for example, decode 0x80 as U+20AC (€) when asked for the "Latin1" or "ASCII" decoding of that byte.

Node.js v23.4.0'a dahil edildi

Node.js v22.13.0'a backport edildi

Node.js v20.18.3'e backport edildi

Her şey harika...



Issue'lar Açılmaya Başladı!

#56219 - "TextDecoder regressed for ascii, windows-1252"
#56542 - "TextDecoder incorrectly decodes 0x92"
#59515 - "error for Windows-1252 encoding for char \u2013"
#60888 - "TextDecoder behavior is wrong since 20.18.3"

Kullanıcılar:

"0x80 byte'ı € (Euro) yerine yanlış karakter veriyor!"
"0x92 byte'ı ' (smart quote) yerine bozuk!"



Issue

**[https://github.com/nodejs/node/
pull/60889](https://github.com/nodejs/node/pull/60889)**

Sorunu çözen pr

<https://github.com/nodejs/node/pull/60893/files>

Teşekkürler